## Objective

Design and implement an enterprise-grade AI Assistant capable of answering questions from organizational knowledge sources while demonstrating modern agentic AI patterns, observability, security controls, and production-ready engineering practices.

The solution should showcase your ability to design scalable AI systems rather than simply connecting an LLM to a vector database.

Can use any AI tools , but heavily checking the explainability and surrounding logic with code readability .

Try to cover as much as possible picking up what is matter for POC first with given time .
Maintain git commit history on the repo.
You may have many questions in the middle , you can make assumptions on any question by picking the best path as you see .

# Business Scenario

A company maintains thousands of internal documents including:

- Policies
- Architecture documents
- Runbooks
- Incident reports
- Product specifications
- Meeting notes

Employees need a conversational assistant that can:

- Search across documents
- Answer questions
- Explain reasoning
- Retrieve supporting evidence
- Maintain conversation context
- Invoke external tools when required

The platform must support multiple user roles with controlled access to information and tools.

# Technical Requirements

## Frontend

Implement a lightweight chat interface using:

- Streamlit

UI beauty is not important.

Focus on functionality and transparency.

### Required UI Features

#### Chat Window

- Multi-turn conversations
- Streaming responses

#### Agent Activity Panel

Display in real time:

- Current agent state
- Active node in LangGraph(Deep agents)
- Tool calls being executed
- Retrieval status
- Memory updates
- Validation results
- Final response generation

The evaluator should be able to observe what the agent is doing internally.

# Backend

Implement using:

- Python
- FastAPI
- Async programming patterns

Requirements:

- Async APIs
- Async retrieval
- Async tool execution
- Proper exception handling
- Structured logging

# AI Architecture

## LangGraph

Use LangGraph as the orchestration engine. The

graph should include multiple specialized agents.

Example only:

### Supervisor Agent

Responsible for:

- Intent understanding ● Task decomposition ● Agent routing

### Retrieval Agent

Responsible for:

- RAG operations
- Vector search

### Research Agent

Responsible for:

- Deep investigation
- Recursive exploration

### Response Agent

Responsible for:

- Final answer generation

# Recursive Language Model (RLM)

The solution must demonstrate Recursive Language Model concepts.

Rather than loading entire documents into the LLM context:

The agent should:

1. Explore document collections
2. Generate Python-based search plans
3. Decompose large tasks
4. Retrieve targeted sections
5. Call sub-agents recursively
6. Aggregate results

Example:

User asks:

> Summarize all outage reports related to payment failures during the last year and identify recurring root causes.

Expected behaviour:

- Search relevant documents
- Filter by topic
- Break into batches
- Analyze batches separately
- Aggregate findings
- Produce final summary

The candidate may implement a simplified version but should demonstrate the concept.

# Retrieval Architecture

Implement hybrid search.

## Dense Search

Using embeddings.

## Sparse Search

Keyword/BM25 search.

## Hybrid Ranking

Combine:

- Dense score
- Sparse score

# Vector Database

Use Pinecone.

Requirements:

- Namespaces
- Metadata filtering
- Hybrid retrieval
- Document attribution

Metadata examples:


```
{
 "department": "payments",
 "document_type": "incident",
 "access_level": "internal",
```

"created_date": "2025-01-01"
}

# Memory

Implement conversational memory.

Should maintain:

- User context
- Previous questions
- Relevant historical interactions

Memory should survive multiple turns during a session.

Explain memory design decisions.

# Tool Calling

At minimum implement tools for:

### Knowledge Search Tool

Search indexed documents.

### MCP Tool

Create a simple MCP server exposing dummy enterprise data.(not a high priority requirment)

Example:

- Employee directory
- Service catalog
- Incident records

The agent should invoke MCP tools when required.

### Python Analysis Tool

Perform structured analysis on retrieved data.

# LLM

Any modern LLM may be used.

Examples:

- OpenAI
- Anthropic
- Gemini
- Open-source alternatives

Model selection rationale should be documented.

# Observability

## LangSmith

Mandatory.

Requirements:

- Trace every conversation
- Trace tool calls
- Trace agent transitions
- Trace retrieval operations

The evaluator should be able to inspect execution traces.

# Security Requirements

## Prompt Injection Protection

Implement protections against:

- Instruction override attempts
- Data exfiltration attempts
- Tool abuse attempts

Document the approach.

## Input Validation

Validate:

- User requests
- Tool parameters
- Retrieved content

## Guardrails

Implement guardrails for:

- Unsafe tool execution
- Unauthorized access
- Hallucinated citations
- Invalid responses
  - Consider about the brand value (you can use commercial bank as the bot own company)

# Authentication and Authorization

Implement one of:

### Option A

Hardcoded users and roles

or

### Option B

Open-source authentication solution

Examples only:

- Keycloak

# Role Based Access Control

Required roles:

## Viewer

Allowed:

- Chat
- Search

Not allowed:

- Administrative tools

## Analyst

Allowed:

- Search
- Analytics tools
- MCP tools

## Administrator

Allowed:

- All tools

Tool execution must respect role permissions. The

agent should not be able to bypass authorization.

# Rate Limiting

Implement Token Bucket rate limiting.

Requirements:

- Per-user limits
- Configurable thresholds
- Graceful error handling

# Error Handling

Demonstrate handling for:

- LLM failures
- Vector DB failures
- MCP failures
- Tool timeouts
- Invalid requests

The application should degrade gracefully.

# Sample Documents

Candidate may create mock data including:(you can generate this)

- Incident reports
- Architecture documents
- Operational runbooks
- Product specifications

# Evaluation Criteria

| Area | Weight |
|---|---|
| Agent Architecture | 20% |
| RAG Design | 15% |
| LangGraph Usage | 15% |
| RLM Implementation | 10% |
| Security & Guardrails | 10% |
| Observability | 10% |
| Async Engineering | 5% |
| RBAC | 5% |
| Code Quality | 5% |
| Documentation | 5% |

# Bonus Points

- Multi-agent collaboration (way of agents manage the state and how failure getting handle , butterfly effect )
- Human-in-the-loop approval node
- Reranking layer
- Long-term memory
- Feedback loop for answer quality
- Containerized deployment (Docker Compose)

# Deliverables

1. Source code repository need to be public
2. Architecture diagram
3. Demo video (45 minutes): public URL
4. LangSmith traces in the demo
5. Assumptions and trade-offs in the demo